

CONTAINER-DISTRIBUTION

HelixTranslate container distribution (thinker.local + amber.local)

Per the mandate that **HelixTranslate and other infrastructure/services run ONLY via the Containers submodule, distributed to thinker.local and amber.local** (ssh milosvasic, key-based), all translation compute runs inside containers on those two hosts. No local engine binary is used.

Hosts

host	OS	runtime	role
thinker.local	Ubuntu 24.04 (6.17)	podman	cgo image BUILD host + translate worker
amber.local	Ubuntu (6.8)	docker	translate worker (image replicated to it)

Image: helixtranslate:cli

Multi-stage build (_tools/helixtranslate-container/Containerfile.translator): go lang:1.26-alpine + gcc musl-dev sqlite-dev → go build ./cmd/unified-translator, runtime alpine:3.20 + ca-certificates sqlite-libs. ~20 MB.

cgo is required. The bridge opens a go-sqlite3 verified-models store at startup (./data/verified_models.db); a CGO_ENABLED=0 build ships a stub that fatals on ping (go-sqlite3 requires cgo to work). Verified empirically — the CGO-free image fails, the cgo image (Unified Translator v2.3.1) translates.

Verified-models store (the second gate)

Even with an explicit -provider, the bridge runs an LLMsVerifier challenge at startup and refuses to translate if no model meets the threshold (verified+can_see_code+affirmative). On an empty db this verification runs for ~5 min against live APIs and then fails. Fix: seed verified_models.db (7 verified models, from the local working repo) into a persistent helixtranslate-data volume mounted at /data. With a non-empty store needVerify is false → bridge.Open returns early → the explicit ClientForProvider(mistral) path translates directly in ~37 s.

Secrets

LLM keys live only in ~/.helixtranslate.env (mode 600) on each host, loaded via <runtime> run --env-file. Trimmed to the providers with balance (MISTRAL/GROQ/COHERE). No key is passed on a command line or logged.

Execution path

```
translate-fleet.sh                                # parallel driver,
round-robins hosts
└─ translate-pipeline.sh (--article)              # frontmatter split,
retry+fallback, artifact strip
└─ HELIX_TRANSLATE_BIN=helixtranslate-container.sh #
engine-compatible shim
└─ ssh milosvasic@<host> run.sh <rt> <provider>
<model> <sl> <tl> <script>
└─ <podman|docker> run --rm -i --env-file
~/.helixtranslate.env \
-v helixtranslate-data:/data
helixtranslate:cli
```

Source streams in on stdin; only the translated markdown returns on stdout (engine logs to stderr). Output is written atomically and only on success.

Reproduce

`_tools/distribute-helixtranslate.sh` performs the whole flow (rsync source → build on thinker → replicate to amber → seed db → install env/run.sh) and is re-runnable.

Result

316 article translations (milosvasic.ru ×12 slugs, vasic.digital ×18 slugs, into be/de/es/fr/zh/kk/hi/ja/ko/ar/tr/fa) produced entirely on the two hosts, then independently reviewed (Cohere, §11.4.141) and deployed.